$ SYSTEM.SHELL

Shell in the Operating System

The command-line bridge between the user and the kernel

```
user@os:~$ whoami --role interpreter
```

Operating Systems · Command Interpreters

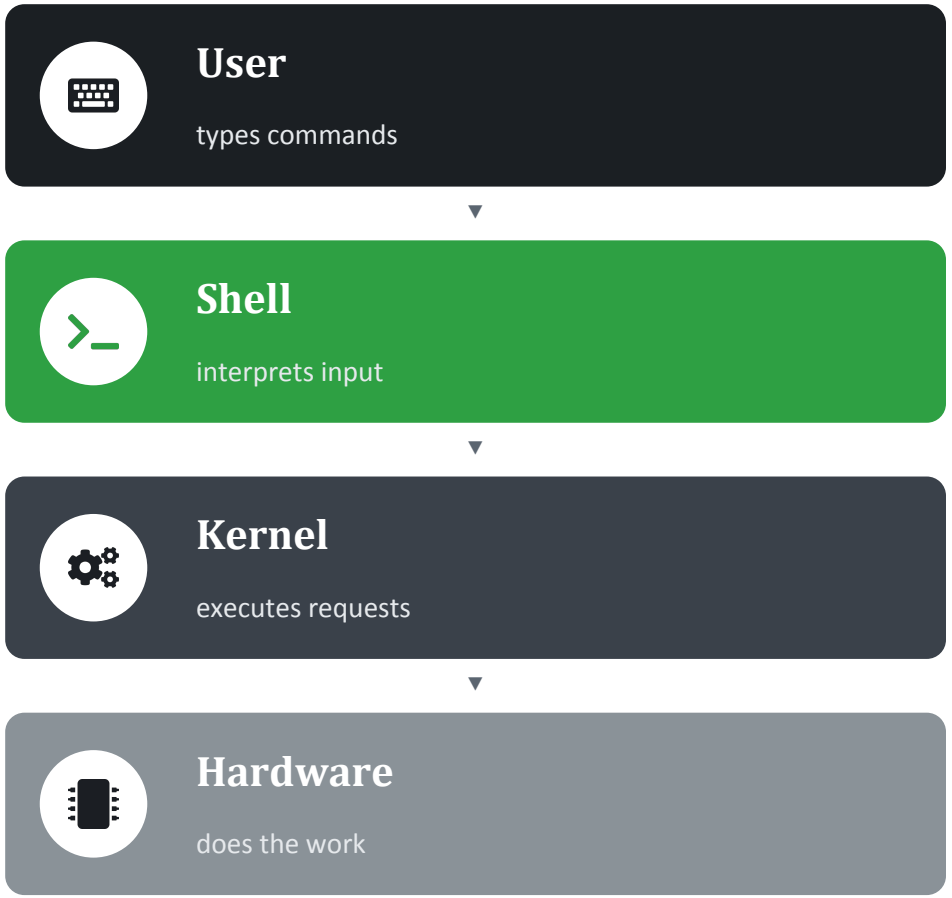
What is a Shell?

A shell is a program that acts as a command interpreter — it takes instructions typed by the user (or from a script) and translates them into actions the operating system's kernel can perform.

It sits as an outer layer around the kernel, giving users a way to access system services without writing low-level code.

`bash` `zsh` `PowerShell` `csh`

Common shell programs



Types of Shells

Shells are broadly classified by how the user interacts with them:

Command-Line Shell (CLI)

The user types text commands at a prompt, and the shell interprets and executes them. Fast, scriptable, and the standard for system administration.

- Bash
- Zsh
- Csh
- PowerShell

Interaction: typed commands

Graphical Shell (GUI)

The user interacts through windows, icons, and menus using a mouse or touch, instead of typed commands. Easier to learn for everyday, non-technical use.

- Windows Explorer
- GNOME Shell
- KDE Plasma

Interaction: mouse & visual controls

Some environments also offer a restricted shell — a limited CLI that blocks certain commands for security.

Popular Shells Comparison

Comparison between some of the most popularly used shells across various operating systems:

Shell	Features	Used In
`Bash`	Command history, scripting, job control	Linux, macOS
`Zsh`	Auto-completion, themes, plugins	Power users on Linux/macOS
`Csh`	C-like syntax	Older Unix systems
`Fish`	User-friendly, syntax highlighting	Linux

Why Do We Need a Shell?

Without a shell, users would need to interact with the kernel directly through raw system calls — the shell provides a safe, convenient, and standardized interface instead.

User–Kernel Interface

Provides a controlled gateway between the user and sensitive kernel operations.

Efficient Task Execution

Commands run instructions quickly without needing a full application built for it.

Automation via Scripting

Repetitive tasks can be automated by chaining commands into shell scripts.

System Management

Enables file handling, process control, and configuration from one place.

Program Combination

Pipes and redirection let small tools combine into powerful workflows.

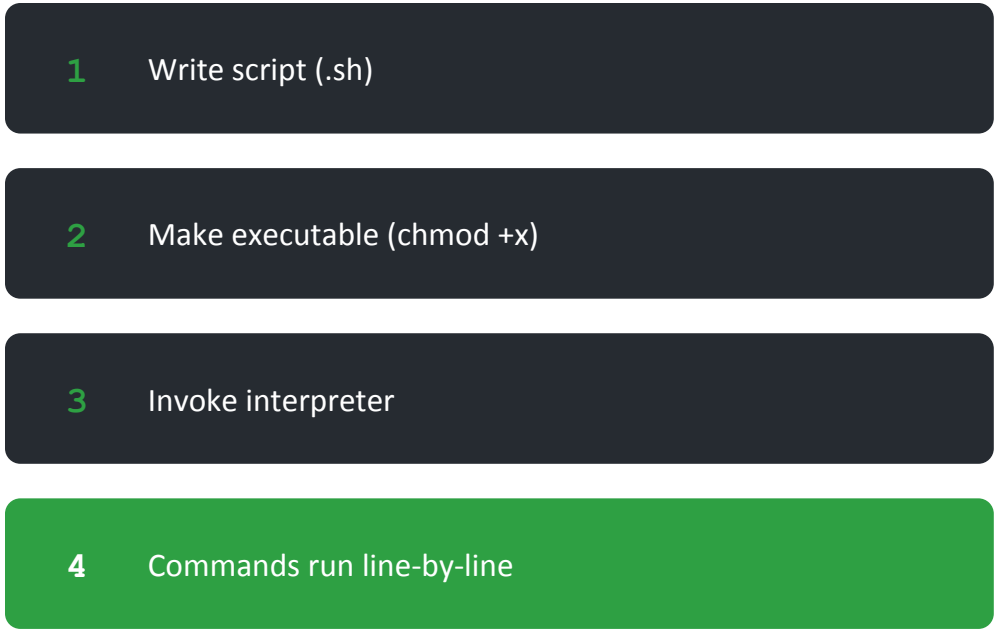
Controlled Access

Restricts users to permitted operations, protecting the kernel from misuse.

Shell Scripting: The Basics

A shell script is a text file containing a sequence of shell commands, executed line by line by the shell interpreter — used to automate repetitive or complex tasks.

- **Shebang line** — #!/bin/bash — tells the OS which interpreter to use.
- **Variables** — Store data: name="OS" then reference with $name.
- **Control structures** — if/else, for, while loops direct program flow.
- **Input & output** — read takes input; echo prints output to the terminal.
- **Comments** — Lines starting with # are ignored by the interpreter.



Typical script lifecycle, from file to execution

A Simple Shell Script Example

The script below checks whether a number entered by the user is even or odd — a classic first example of variables, input, and conditionals.

evenodd.sh

```
#!/bin/bash

echo "Enter a number:"
read num

if [ $((num % 2)) -eq 0 ]; then
    echo "$num is Even"
else
    echo "$num is Odd"
fi
```

Shebang

Selects bash as the interpreter for this file.

read

Captures user input into the variable num.

$((...))

Arithmetic expansion computes num % 2.

if / else / fi

Branches output based on the condition's result.

Setup Permission & Run a Script

Setup Executable Permission

Once a script is created, it needs executable permission before it can be run. Without it, running the script is almost impossible — and it must also have read permission.

SYNTAX

```
$ chmod +x your-script-name
```

```
$ chmod 755 your-script-name
```

755 = read + write + execute for the owner, read + execute for group and others.

Run the Script

Once the script has proper executable permission, test it by running it in any of these ways:

- `bash your-script-name`
- `sh your-script-name`
- `./your-script-name`

EXAMPLE

```
$ bash bar
$ sh bar
$ ./bar
```

From Script to Execution: A Walkthrough

Putting it all together — write a script with vi, save it, make it executable, then run it.

1 Create the script

```
$ vi first
```

```
#
# My first shell script
#
clear
echo "This is my First script"
```

2 Make it executable, then run it

```
$ chmod 755 first
$ ./first
```

terminal — output

```
$ vi first
$ chmod 755 first
$ ./first
This is my First script
```

The script clears the screen, then prints its message to the terminal.

Components of a Shell

01

Prompt

A symbol or message indicating the shell is ready to accept input.

02

Input / Command Parser

Interprets the entered command before it is executed.

03

Execution Environment

Interacts with the kernel to execute commands.

04

Scripting Capabilities

Allows users to write shell scripts for automation.

Functions of a Shell

Command Execution

Accepts and runs commands entered interactively or from scripts.

File & Directory Handling

Navigate, create, move, and manage files through simple commands.

I/O Redirection

Redirects standard input/output/error to files or other programs.

Process Management

Creates, schedules, and terminates processes on the user's behalf.

Scripting & Automation

Executes saved sequences of commands to automate repetitive work.

Program Interconnection

Uses pipes to pass output of one program as input to another.

How a Shell Runs a Command

Every time you type a command, the shell repeats the same three-step cycle using three system calls:

1

`fork()`

The shell clones itself. Now two processes exist: the original shell (parent) and an identical copy (child).

▸

2

`exec()`

The child replaces itself with the requested program (e.g. ls). Same PID, new code running.

▸

3

`wait()`

The parent shell pauses until the child finishes, then reaps it and prints the next prompt.

This cycle repeats for every command — it's the same pattern behind the Execution Environment component and Command Execution function covered earlier.

fork() & wait(): One Process Becomes Two

fork() runs once but returns twice — once in each process — with a different value telling each side who it is.

Where	Return value	Meaning
Parent	Child's PID	"I'm the parent"
Child	0	"I'm the child"
Failure	negative	fork failed

Both branches run — just in two different processes. Which one gets the CPU first is not guaranteed.

warm_up_1_wait.c — the fix

```
pid_t pid = fork();
if (pid == 0) {
    printf("I am child...");
} else {
    wait(NULL);
    printf("I am parent...");
}
```

OUTPUT ORDER

Without wait() — order varies

```
I am parent. PID = 1150272
I am child.  PID = 1150273
```

With wait() — child always first

```
I am child.  PID = 1150977
I am parent. PID = 1150976
```

wait(NULL) blocks the parent until the child finishes — and reaps it, preventing a zombie process.

The exec() Family

exec() doesn't create a new process — it replaces the current one's code in place. On success it never returns; the old program is simply gone.

Function	Arguments	PATH lookup	Example
execl	Listed individually	Needs full path	`execl("/bin/ls", "ls", "-l", NULL)`
execlp	Listed individually	Searches $PATH	`execlp("ls", "ls", "-l", NULL)`
execvp	Passed as an array	Searches $PATH	`execvp("ls", args)`

Memory trick: **`l` vs `v`** = how arguments are passed (**l**ist vs **v**ector) · trailing **`p`** = searches **`$PATH`** for you

Advantages & Disadvantages

Advantages

- Automates repetitive tasks via scripting
- Lightweight and fast for direct system access
- Highly flexible — combines small tools via pipes
- Works well over remote/low-bandwidth connections
- Portable scripts across Unix-like systems

Disadvantages

- Steep learning curve for beginners
- Limited error handling compared to full languages
- Slower execution than compiled programs
- Syntax varies across shells (bash, zsh, csh)
- Not ideal for complex application logic

$ SUMMARY

The Shell: Simple Interface, Powerful Control

From typing a single command to automating entire workflows, the shell remains one of the most direct ways to work with an operating system.

```
user@os:~$ exit 0
```